TP5: MPI Point-to-Point and Collective Communications

Khaoula Mejhoudi

February 2025

1 Exercise 1: Hello World

The goal was to write a basic MPI program. Each process prints its rank and the total number of processes. Only rank 0 prints an extra message.

```
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
printf("I am process %d among %d\n", rank, size);
if (rank == 0) printf("I'm here alone\n");
```

Omitting `MPI_Finalize()` causes MPI resources to not be released properly. In practice, most implementations still terminate, but it is undefined behavior and can cause issues in some environments.

2 Exercise 2: Sharing Data

Process 0 reads an integer and broadcasts it to all processes using `MPI_Bcast`. This repeats until a negative value is entered.

`MPI_Bcast` is a collective operation: every process must call it, but only the root (rank 0) provides the data. All others receive it.

```
do {
    if (rank == 0) scanf("%d", &v);
    MPI_Bcast(&v, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received %d\n", rank, v);
} while (v >= 0);
```

3 Exercise 3: Sending in a Ring

Process 0 reads a value, sends it to process 1. Each process receives the value, adds its rank, prints the result, and forwards it to the next process.

The communication pattern uses only `MPI_Send` and `MPI_Recv`:

- Process 0: reads, sends to rank+1 (no receive).
- Middle processes: receive from rank−1, add rank, print, send to rank+1.
- Last process: receives, adds rank, prints (no send).

Process 0 sends before any receive happens, which prevents deadlock since the ring flows in one direction.

4 Exercise 4: Matrix-Vector Product

4.1 Implementation

The matrix A (size $N \times N$) is distributed row-by-row across processes using `MPI_Scatterv`. The vector b is sent to all processes using `MPI_Bcast`. Each process computes its portion of the result, and results are collected with `MPI_Gatherv`.

`MPI_Scatterv` (instead of `MPI_Scatter`) is used to handle cases where N is not divisible by the number of processes. The first $N \mod P$ processes get one extra row.

The parallel workflow is:

$$\text{Bcast}(b) \;\rightarrow\; \text{Scatterv}(A) \;\rightarrow\; \text{local multiply} \;\rightarrow\; \text{Gatherv}(x)$$

4.2 Results

N	$P = 1$ (s)	$P = 2$ (s)	$P = 4$ (s)	$P = 8$ (s)	Speedup ($P = 4$)
1000	0.0074	0.0067	0.0071	0.0213	1.04
4000	0.1250	0.1203	0.1247	0.2030	1.00
8000	0.5170	0.5142	0.5036	0.5537	1.03
10000	0.7949	0.7403	0.8033	0.8779	0.99

Table 1: Execution times for matrix-vector multiplication

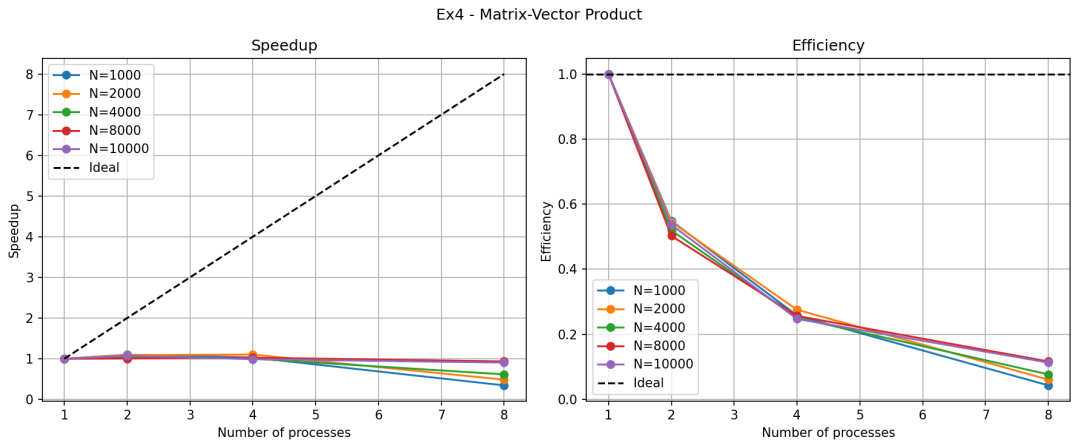

Figure 1: Speedup and Efficiency: Matrix-Vector Product

4.3 Analysis

The speedup is very close to 1 regardless of the number of processes. This is expected: the bottleneck is communication, not computation. `MPI_Scatterv` must send $O(N^2)$ data to each process, which is the same cost as the computation itself. The communication overhead cancels out any gain from parallelism. With 8 processes the program is even slower due to synchronization overhead.

For this type of problem, MPI parallelism only becomes worthwhile at very large N where computation time dominates communication time.

5 Exercise 5: Pi Calculation

5.1 Implementation

π is approximated using:

$$\pi \approx \frac{4}{N} \sum_{i=0}^{N-1} \frac{1}{1+x_i^2}, \quad x_i = \frac{i+0.5}{N}$$

Each process handles every P-th iteration (interleaved distribution): process r computes $i = r, r+P, r+2P, \ldots$ This naturally handles cases where N is not divisible by P without any special logic.

Each process computes a local partial sum. `MPI_Reduce` with `MPI_SUM` collects all partial sums into the final result on rank 0.

5.2 Results

N	$P = 1$ (s)	$P = 2$ (s)	$P = 4$ (s)	$P = 8$ (s)	Speedup ($P = 4$)
10^6	0.0037	0.0019	0.0011	0.0013	3.51
10^7	0.0331	0.0181	0.0088	0.0094	3.76
10^8	0.3436	0.1716	0.1090	0.1096	3.15
10^9	3.4725	1.7525	0.8823	1.0839	3.94

Table 2: Execution times for Pi calculation

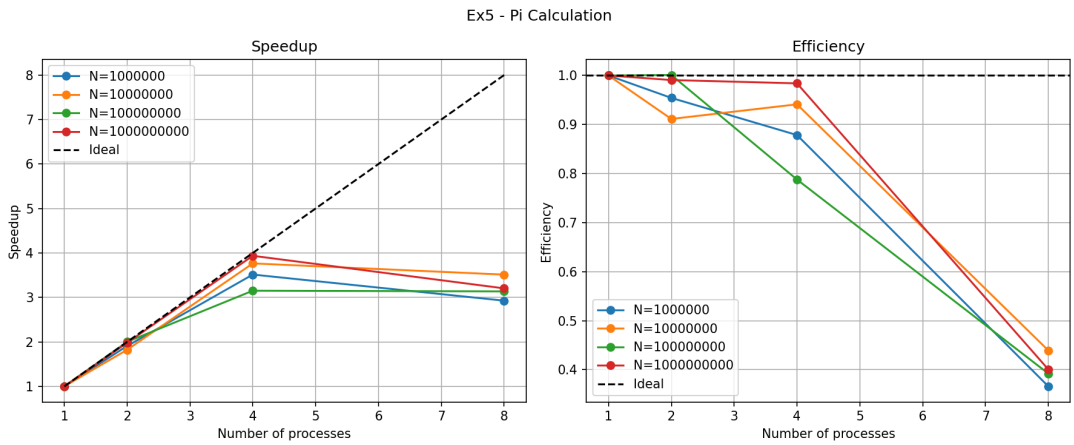

Figure 2: Speedup and Efficiency: Pi Calculation

5.3 Analysis

Pi calculation shows near-linear speedup with 4 processes (speedup ≈ 3.94 for $N = 10^9$, efficiency $\approx 98\%$). This is because each iteration is independent and the only communication is a single `MPI_Reduce` call at the end, so the communication cost is $O(1)$ regardless of N.

With 8 processes, performance drops slightly compared to 4 (speedup ≈ 3.20). On this cluster, the benefit of 8 processes is limited likely due to shared resources or NUMA effects on the login node.

This exercise illustrates a perfectly parallel workload: no data dependencies, minimal communication. It is a good example of Amdahl's law: since there is almost no sequential part, the speedup scales nearly linearly with the number of processes.

6 Conclusion

This lab covered the two main types of MPI communication:

- **Point-to-point** (`MPI_Send`/`MPI_Recv`): used in the ring exercise. Simple but requires careful ordering to avoid deadlock.
- **Collective** (`MPI_Bcast`, `MPI_Scatter`/`Gather`, `MPI_Reduce`): used in exercises 2, 4, and 5. More expressive and often better optimized by the MPI library.

The key lesson from the performance results is that **communication overhead matters**. Pi calculation parallelizes well because communication is minimal ($O(1)$). Matrix-vector multiplication does not scale here because the cost of distributing the matrix ($O(N^2)$) equals the computation cost, leaving no room for speedup.